When Every Row Counts: Verifier-Guided Agentic Synthesis for Small Tabular Data

Sancho Amaral Simões[1,*], João P. Vilela[1,2], Pedro Henriques Abreu[1]
[1]*CISUC-DEI - FCTUC, University of Coimbra*, Coimbra, Portugal
[2]*CRACS/INESCTEC-DCS - FCUP, University of Porto*, Porto, Portugal
[*]Corresponding author
sanchosimoes@student.dei.uc.pt, jvilela@fc.up.pt, pha@dei.uc.pt

***Abstract*—Many prediction tasks in medicine must be learned from small tables: a clinical cohort or a disease registry often holds a few hundred labelled records, and its rarest class only a few dozen. We call a table *small* when it has at most 1,000 rows. Synthetic rows could enlarge such tables, but deep generators (GANs, variational autoencoders, diffusion models) must first be trained on the very rows that are scarce. Large language models (LLMs) need no training and can write new rows from a few real examples placed in the prompt, yet their raw output is not a usable table: it can contain too few rows, missing or invented columns and values of the wrong type, and its cost is rarely reported. We present SYNTHAGENT, a system that treats every LLM response as an unverified proposal. A deterministic controller decides which real rows the model sees, repairs and validates each response against the table schema, requests the missing rows until the table reaches the requested size, and logs the cost and every correction. We compare SYNTHAGENT with SMOTE, CTGAN and TVAE on six medical tables of 195 to 768 rows, over ten repeated splits and ten classifiers. SYNTHAGENT fills 99.7% of the cells it is asked for, keeps a membership-inference attack near chance (AUC 0.53) and, with GPT-4.1, gives the largest mean F1 gain of the five generators ($+1.1$ points). No generator, however, improves utility significantly on every dataset, and repeated runs of the same pipeline already differ by up to 0.6 points, a threshold below which design differences should not be interpreted.**

***Index Terms*—synthetic data, small data, tabular data generation, large language models, data quality, data augmentation, healthcare data**

I. INTRODUCTION

Context: small tabular data: Many prediction tasks in medicine are learned from tables with a few hundred records: the cohort of a single hospital, a disease registry or a pilot study. Collecting more records is slow and expensive, and often impossible when the condition is rare or the data cannot leave the institution. Throughout the paper we use the following working definition.

Definition 1 (Small tabular dataset). A labelled table with n rows, d features and $n_{\min}$ rows in its least frequent class is *small* when $n \leq 1{,}000$. We characterize it further by its events per variable, $\mathrm{EPV} = n_{\min}/d$.

The threshold is two orders of magnitude below the tables on which deep tabular generators are usually benchmarked, such as Adult (48,842 rows) or Covertype (581,012 rows) [1]. At this size every record carries at least 0.1% of the evidence, a 20% test split removes up to 200 rows, and the least frequent class has at most a few hundred examples. EPV links the definition to clinical practice: sample-size guidance for clinical prediction models asks for at least 10 minority-class events per candidate feature, and often many more [2], [3]. The six tables we study have 195 to 768 rows and an EPV between 1.0 and 33.5, and four of them fall below 10 (Table III).

Problem and why it matters: Synthetic data generation produces artificial rows that follow the distribution of a real table. For a small table it promises more training examples, more examples of a rare class, and data with which to build and test pipelines without exposing patient records. Small tables are, however, exactly where generation is hardest. First, a learned generator must estimate the joint distribution of all columns from the same few rows it is meant to supplement; in our experiments, only 29.6% of CTGAN rows stay inside the observed schema (Section VI-E). Second, a small test set makes it hard to tell whether synthetic rows helped at all: across ten random splits of the same table, the accuracy of a model trained on real data alone has a standard deviation of up to 18 points (Section VI). The problem this paper addresses is therefore twofold: *how to generate synthetic rows for a small table without training a generator on it, while guaranteeing a valid table of the requested size; and how to measure reliably whether those rows help a downstream model.*

Large language models (LLMs) are a natural answer to the first half. They need no per-table training, they know what column names such as `age` or `cholesterol` mean, and they can imitate a few dozen real rows shown in the prompt [4], [5]. An LLM response, however, is a *proposal*, not a table. Nothing obliges it to return the requested number of rows, well-formed JSON, the declared columns, or values of the declared types (Figure 1, left), and each request costs tokens and time that nobody accounts for.

Existing approaches and their limitations: Table I groups prior generators by what they guarantee. Interpolation methods such as SMOTE [6] need no training but only mix existing minority rows and ignore column types. Deep generators such as CTGAN, TVAE [1], TabDDPM [7] and TabSyn [8] learn the joint distribution, which requires the large samples that small tables lack. Fine-tuned language models such as GReaT [4] and REaLTabFormer [9] also train on the target table. Prompted LLMs such as CLLM [5] and EPIC [10] avoid training and target the low-data regime, but they focus on what

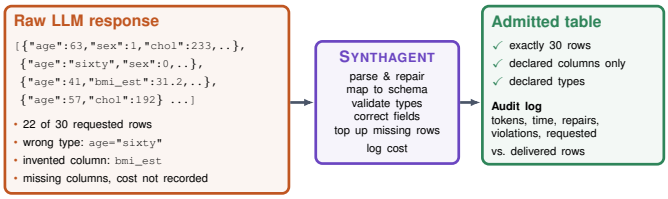


Fig. 1. The problem in one request. Asked for 30 rows of a heart-disease table, a raw LLM response (left) can return too few rows, an invented column, a missing column or a value of the wrong type. SYNTHAGENT (right) checks and corrects every response before admitting it, so the table that reaches the user has the requested size and schema, and a log records what was repaired and at what cost.

TABLE I
POSITIONING OF SYNTHAGENT AGAINST GENERATOR FAMILIES. ✓: PROVIDED BY DESIGN; (✓): PARTIALLY, E.G. UNPARSABLE SAMPLES ARE DISCARDED; ×: NOT ADDRESSED.

Family	No table training	Column semantics	Schema checked	Exact size	Cost logged
Interpolation [6]	✓	×	×	✓	×
Deep gen. [1], [7], [8]	×	×	×	✓	×
Fine-tuned LMs [4], [9]	×	✓	(✓)	✓	×
Prompted LLMs [5], [10]	✓	✓	×	×	×
SYNTHAGENT (ours)	✓	✓	✓	✓	✓

to put in the prompt and which generated rows to keep; they do not guarantee that the output is a well-formed table of the requested size, and they do not report its cost. On the evaluation side, generators are usually compared from single runs, so a difference of one point between two configurations is reported as an effect even when two runs of the same configuration differ by as much.

Our approach and what is different: We present SYNTHAGENT, which separates *proposing* rows, done by the LLM, from *admitting* them, done by a deterministic controller. The controller keeps an explicit record of the schema, the real rows shown, the rows accepted so far and the rows still missing, and each of its four components removes one observable failure: a context budgeter chooses which real rows fit in the prompt; a parser repairs malformed output and maps returned columns back to the schema; a validator checks column types and, when a row fails, asks the model to correct only the offending fields; and a top-up loop requests exactly the missing rows until the table has the requested size. Every token, repair and violation is logged, so each synthetic table carries its own audit trail. Compared with the families in Table I, SYNTHAGENT needs no training on the small table, unlike deep and fine-tuned generators, and it guarantees schema, size and accounting, which prompted-LLM methods leave to the model. We also change how such systems are evaluated: every augmented model is paired with a real-only model on the same split, and we measure how much two runs of the *same* pipeline differ, the *noise floor*, before attributing any difference to a design choice.

Contributions:

1) **SYNTHAGENT, an open-source control layer for LLM-based synthesis of small tables** that guarantees schema-checked output of the requested size and records its cost and repairs (Section IV).
2) **An evaluation protocol for small data** that pairs every augmented model with a real-only model on the same split, reports fidelity, privacy diagnostics, utility and cost together, and calibrates comparisons with an empirical noise floor (Sections V and VI-C).
3) **An empirical study on six medical tables** with four LLM backends and three non-LLM generators (SMOTE, CTGAN, TVAE), 780 dataset-configuration repetitions and up to 23,400 classifier fits, reporting null and negative results alongside positive ones (Section VI).

Main findings: (i) SYNTHAGENT returns complete tables (99.7% of cells filled) whose fidelity is stable across all control variants. (ii) The choice of LLM is the only factor whose effect clearly exceeds the noise floor: GPT-4.1 gives the largest mean F1 gain of all generators ($+1.09$ points). (iii) No generator dominates: TVAE gives the best mean accuracy change at a fraction of the cost, and SMOTE helps most when the positive class is rare. (iv) No configuration improves utility significantly on every dataset, so synthetic rows are not a substitute for real evidence; SYNTHAGENT makes this trade-off measurable before augmented data is used.

Paper organization: Section II reviews related work, Section III formalizes the problem, Section IV presents SYNTHAGENT, Section V describes the experimental protocol, Section VI answers five research questions, Section VII discusses limitations and Section VIII concludes.

II. RELATED WORK

SYNTHAGENT sits where data engineering, generative modeling and LLM-based data processing meet. We review each line of work in turn and close each paragraph with how our system differs.

Synthetic data in data engineering: Synthetic data has a long history in data management, where it is used to test systems, share data and protect privacy. Sanghi and Haritsa survey workload-aware synthetic database generation for enterprise systems [11]. Fan et al. explore the design space of GAN-based relational synthesis and its privacy-utility trade-off [12]. Differentially private synthesis releases data through noisy probabilistic models such as Bayesian networks [13]; Kamino additionally enforces denial constraints during private synthesis [14], a concern that SYNTHAGENT shares through its typed validators. SiloFuse generates cross-silo synthetic data with latent tabular diffusion without centralizing features [15]. These generators are fitted to each table. SYNTHAGENT instead spends inference tokens on an existing instruction-following model, which avoids per-table training on a few hundred rows but makes context, output validity and cost first-class engineering problems.

Deep and language-model tabular generators: SMOTE interpolates minority examples and remains a strong, cheap baseline for imbalance [6]. CTGAN introduced mode-specific normalization and conditional sampling for mixed columns,

alongside the variational TVAE [1]; TabDDPM and CoDi adapt diffusion to mixed types [7], [16], and TabSyn diffuses in a learned latent space [8]. GReaT fine-tunes an autoregressive language model on serialized rows [4]; REaLTabFormer extends this to relational tables [9]; and TabLoft improves LLM-based generation by learning a feature order for serialization [17]. Closest to our setting, CLLM prompts an LLM in the low-data regime and curates the generated rows with learning dynamics [5], and EPIC designs balanced, grouped prompts for imbalanced-class synthesis [10]. These works focus on the quality of the generated distribution. SYNTHAGENT is complementary: it contributes the deterministic control layer (budgeting, parsing, validation, exact-volume completion and audit) that any in-context generator needs to produce a dependable table, and an evaluation protocol that separates real effects from run-to-run noise.

LLMs for data preparation and structured output: Foundation models can perform data cleaning and integration through prompting [18], and data-cleaning systems increasingly combine statistical inference with integrity constraints [19], [20]. Recovering structured records from free-form LLM output is itself a data-engineering problem [21]. SYNTHAGENT applies the same principle to generation: integrity rules are profiled from the schema, LLM output is parsed into typed records, and violations are reported back as grouped, field-level evidence. Data-preparation studies also show that pipeline steps that look beneficial can hurt downstream models or fairness [22], [23]; we adopt their paired, multi-model evaluation stance for augmentation. Feature augmentation adds columns from related tables [24]; SYNTHAGENT augments rows instead.

Evaluation and privacy: Synthcity and SynthEval argue for multi-dimensional evaluation rather than a single fidelity score [25], [26]. Classifier two-sample tests ask whether a classifier can tell real from synthetic rows [27]. Membership inference asks whether an attacker can identify training records [28], and tabular synthesizers can remain vulnerable [29]. Syntactic anonymization models such as ℓ-diversity and t-closeness [30], [31] and differential privacy [32] provide formal guarantees that SYNTHAGENT does not claim; our distance-based attack is a diagnostic only.

Cross-silo and federated healthcare data: Federated learning keeps raw data local and aggregates model updates [33], [34], with secure aggregation protecting individual updates [35]. It is a central pattern for multi-hospital medicine [36], [37], and recent work studies privacy and fairness when models are fused across healthcare federations [38]. Section IV-E shows how SYNTHAGENT can run inside each site's trust boundary.

III. PROBLEM FORMULATION

The related work leaves open what a dependable LLM-based generator must guarantee when only a few hundred rows exist. Before describing the system, we make this setting precise and state the objectives against which every configuration is later judged.

Small-data regime: We work with tables that are small in the sense of Definition 1: at most 1,000 rows, with few minority-class events per feature. Table III lists the six evaluation tables: they have 195 to 768 rows, their least frequent class holds 13 to 268 rows, and their EPV ranges from 1.0 to 33.5. After an 80/20 split, the generator sees a 20% sample of the remaining training rows as context, that is, 31 to 123 real rows.

This setting creates three linked risks. *Coverage risk*: a random context sample can miss a rare class or local mode. *Estimation risk*: apparent augmentation gains can be split variability, especially when many classifiers and configurations are inspected. *Memorization risk*: high similarity can reflect reproduction of scarce records rather than generalization. Our design answers these with context-selection strategies, matched repeated holdouts with corrected tests, and novelty and membership diagnostics.

Synthesis as a constrained multi-objective problem: Let D^{tr} and D^{te} denote a real training and holdout partition, and let a controller configuration θ produce a synthetic table G_θ without observing D^{te}. A valid output belongs to the feasible set $\mathcal{F}(S, R, N)$ induced by schema S, active rules R and requested size N. Choosing a configuration is a multi-objective problem:

$$\theta^\star \in \operatorname*{arg\,min}_{\theta:\, G_\theta \in \mathcal{F}(S,R,N)} \big(L_{\mathrm{fid}},\ L_{\mathrm{priv}},\ -\Delta U,\ \kappa\big)(\theta), \qquad (1)$$

where $L_{\mathrm{fid}}(D^{\mathrm{tr}}, G_\theta)$ is a fidelity loss, $L_{\mathrm{priv}}(D^{\mathrm{tr}}, G_\theta)$ an empirical privacy risk, $\Delta U(G_\theta)$ the downstream utility change and $\kappa(\theta)$ the inference cost. We deliberately do not collapse Equation (1) into a weighted scalar, because the objectives encode different deployment preferences: a hospital sharing data for pipeline testing weights privacy and validity, while a team augmenting a classifier weights utility. The system guarantees feasibility up to explicit bounds; the experiments map the remaining trade-off surface. Section IV describes how feasibility is enforced, and Section V how the four objectives are measured.

IV. THE SYNTHAGENT SYSTEM

This section turns the feasibility requirement of Equation (1) into a concrete control loop. Figure 2 shows the architecture. A profiling stage derives the schema and integrity rules; a context stage selects and budgets real demonstration rows; and a verifier-guided cycle turns each LLM response into parsed, validated and counted records, with a typed recovery action for each failure.

A. Control State

Let a source table be $D = \{x_i\}_{i=1}^n$ with schema S, requested synthetic volume N and optional rules R. The controller maintains

$$z_t = (S,\, R,\, C_t,\, A_t,\, N - |A_t|,\, B_t,\, E_t), \qquad (2)$$

where C_t is the context exposed at step t, A_t the accepted rows, B_t the remaining token budget and E_t the parser and

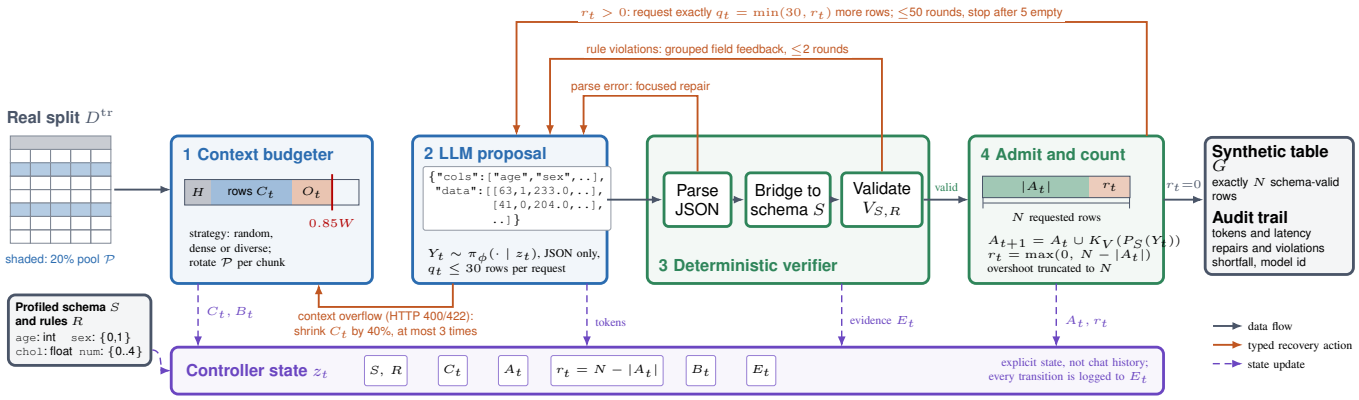


Fig. 2. Architecture of SYNTHAGENT. Real rows enter a token-budgeted context window (1); the LLM proposes a JSON batch (2); deterministic gates parse it, bridge it to the schema and validate typed rules (3); and admitted rows are counted against the requested volume (4). Each failure type triggers its own recovery action (orange), and every step reads or writes the explicit controller state z_t (purple), which doubles as the run's audit trail.

validator evidence. Residual demand $N - |A_t|$ is part of the state rather than something left implicit in a conversation history. The LLM policy π_ϕ proposes a raw batch $Y_t \sim \pi_\phi(\cdot \mid z_t)$. With parser and schema bridge P_S, validator $V_{S,R}$ and a correction operator K that re-prompts with grouped violation evidence for at most two rounds, the accepted-set transition is

$$A_{t+1} = A_t \cup K_{V_{S,R}}\big(P_S(Y_t)\big). \tag{3}$$

Equation (3) separates stochastic proposal from deterministic admission: only output that parses and aligns to S can enter A_{t+1}, and output that fails validation is sent back with targeted evidence before admission. Each transition also updates B_t and appends the failure evidence to E_{t+1}, which becomes the run's audit trail.

B. Context as a Budgeted Resource

The first component of the state that the controller must manage is the evidence the model sees. The context is a controlled information interface, not a dump of the dataset. Table II lists exactly what the model receives in the reported study. The core evidence is a 20% sample of the real training split, serialized as compact row-major JSON with the ordered feature columns and the observed target, so the model sees joint feature-label patterns rather than independent marginal summaries. No free-text description of the clinical endpoint is added: this avoids hand-authored, dataset-specific prompt engineering that would confound the ablation, at the price that the model must infer each column's role from its name and co-occurrence pattern.

Formally, a selection strategy $s \in \{\text{random}, \text{dense}, \text{diverse}\}$ defines an ordered candidate pool, and the token budget determines how much of that pool is visible:

$$\begin{aligned} C_t &= \mathrm{Slice}_t\big(\mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}|\rceil), k_t\big)\,, \\ k_t &= \max\{k : T(H, I, C_{t,k}, O_t) \le 0.85W\}. \end{aligned} \tag{4}$$

Here H is the schema and output contract, I optional instruction text, O_t the reserved output, W the backend context window and T a token estimator (2.8 characters per token for compact JSON). The slice operator rotates through the pool across chunks, so successive requests see different patients when the pool exceeds one window. If the provider still rejects a request as too long (HTTP 400 or 422), the controller shrinks the visible context by 40% and retries, at most three times. "20% context" therefore describes the candidate pool, not a promise that every candidate row appears in every prompt.

The default strategy samples demonstrations at random. *Dense* prefers rows with the most observed cells, reducing ambiguity from missing fields. *Diverse* standardizes numerical features, runs k-means and selects the rows nearest to the centroids, increasing geometric coverage. Neither strategy is class-stratified, so neither guarantees that a rare diagnosis appears in context; Section VII returns to this limitation.

C. Parsing, Validation and Correction

Once a prompt has been sent, the controller must decide which parts of the response can enter the accepted set A_t of Equation (3). Rows are requested in JSON batches of at most 30. A local parser first extracts structured content and fixes common defects (non-finite literals, trailing commas, concatenated objects). If parsing still fails, the controller issues one focused repair request at temperature 0.1 that quotes the error position and surrounding text. Returned column names are then bridged back to the declared schema.

Schema profiling infers integrity rules for every column (types), and optional boundary rules (numerical ranges with 10% tolerance, or categorical domains with at most 20 values). Dynamic Pydantic models validate each batch. The validator groups failures by field and reports one representative bad value per field, so the correction prompt does not grow linearly with repeated errors. Up to two correction rounds are attempted per batch. User-defined post-processing rules (noise, rounding, normalization) can be applied deterministically afterwards.

Available and active controls must be distinguished. In the reported reference configuration, integrity typing is active, boundary enforcement is off, and no user rules are supplied, so the experiment evaluates the default system rather than the

TABLE II
INFORMATION BOUNDARY FOR ONE GENERATION REQUEST IN THE REPORTED STUDY. "NOT SUPPLIED" IS INTENTIONAL: THOSE SIGNALS CANNOT INFLUENCE THE GENERATED ROWS.

Context layer	What the model receives	Why it is included or excluded
Schema	Ordered column names and an exact column-count requirement	Fixes the record interface and prevents invented or omitted fields.
Demonstrations	Compact row-major serialization of a 20% sample of the training split, with feature and target values	Exposes joint patterns, value formats, label prevalence and plausible cross-column combinations without sending the whole local table.
Generation task	Exact number of new rows for the current chunk, at most 30	Turns an open-ended completion into a bounded action whose output count can be verified.
Output contract	JSON only, fixed column order, complete cells, raw numeric values, no extra metadata	Makes parsing deterministic and removes interface failures unrelated to statistical quality.
Dynamic feedback	Parser error location after malformed JSON, or grouped field violations after failed validation	Reveals only the evidence needed for a targeted repair instead of replaying the full history.
Free-text metadata	Not supplied	Prevents unequal, dataset-specific prompt engineering from confounding the comparison.
Test information	No holdout rows, predictions, metrics or classifier identities	Preserves the evaluation boundary and prevents optimization against the test set.
Global statistics	No separately computed means, quantiles, correlations or class quotas	Fidelity must arise from the selected rows; the consequence is weaker protection against rare-mode omission.

strongest constraint policy. Rules are used only by the verifier and are never inserted into the generation prompt, so no fidelity can be attributed to constraints the model did not see.

D. Bounded Exact-Volume Completion

Valid rows are necessary but not sufficient: the table must also have the requested size. LLMs frequently return fewer rows than requested. The controller derives its next request from the accepted state rather than from the conversation:

$$r_t = \max(0, N - |A_t|), \qquad q_t = \min(30, r_t). \tag{5}$$

Equation (5) requests only the residual q_t, and any overshoot is truncated. The loop runs for at most 50 top-up rounds and stops after five consecutive empty or unparsable responses. Successful runs therefore return exactly N rows; persistent provider failure can still end a run short, and that shortfall is recorded rather than hidden. Algorithm 1 summarizes the complete loop.

Audit trail and cost accounting: Every run records prompt, completion and total tokens, LLM generation time, parser repair attempts, rule violations, the requested and delivered row counts and the model identifier. These records make a synthetic table inspectable after the fact and give the cost term $\kappa(\theta)$ in Equation (1) a measured value.

E. Deployment at the Data Boundary

Because the controller needs only a local table, a local schema and an inference endpoint, it can run inside each site of a cross-silo network. Consider K hospitals, each holding a small table D_k for the same prediction problem, with different case mix, coding practice and minority prevalence. Governance constraints prevent pooling raw rows [36], [37]. Each hospital runs the complete SYNTHAGENT loop inside its own trust boundary and trains locally,

$$G_k = \mathcal{A}(D_k, S_k, R_k, B_k), \qquad \Delta w_k = \mathcal{U}(w_t, D_k \cup G_k), \tag{6}$$

Algorithm 1 Verifier-guided synthesis loop

Require: table D^{tr}, schema S, rules R, volume N, strategy s, window W
$\mathcal{P} \leftarrow \mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}| \rceil)$; $A \leftarrow \emptyset$; $E \leftarrow [\,]$
for chunk $t = 1, \ldots, \lceil N/30 \rceil$ **do**
  $C_t \leftarrow$ rotated slice of $\mathcal{P}$ that fits $0.85W$ ▷ Eq. (4)
  $Y \leftarrow \pi_\phi(S, C_t, \min(30, N - 30(t-1)))$; on overflow shrink C_t by 40%, retry ≤ 3
  $X \leftarrow P_S(Y)$; **if** parse fails **then** $X \leftarrow P_S(\mathrm{repair}(Y))$
  for $j = 1, 2$ **do** ▷ targeted correction
    if $V_{S,R}(X)$ has no violations **then break**
    $X \leftarrow P_S(\pi_\phi(\text{grouped violations of } X))$; append to E
  end for
  $A \leftarrow A \cup X$
end for
$u \leftarrow 0$
while $|A| < N$ **and** fewer than 50 rounds **and** $u < 5$ **do** ▷ Eq. (5)
  $X \leftarrow P_S(\pi_\phi(S, \text{first 20 rows of } \mathcal{P}, \min(30, N - |A|)))$
  if $X = \emptyset$ **then** $u \leftarrow u + 1$ **else** $A \leftarrow A \cup X$; $u \leftarrow 0$
end while
return first N rows of A, together with E, token counts and timings

where S_k and R_k are site-specific schema and policy constraints, B_k the local context and inference budget and $\mathcal{U}$ the client training procedure. Only the update Δw_k and approved aggregate diagnostics leave the site. A FedAvg-style coordinator [33] then aggregates

$$w_{t+1} = w_t + \sum_{k \in \mathcal{K}_t} \frac{n_k}{\sum_{j \in \mathcal{K}_t} n_j} \Delta w_k, \qquad n_k = |D_k|, \tag{7}$$

preferably under secure aggregation [35]. Weighting by the *real* count $|D_k|$ rather than $|D_k \cup G_k|$ gives a site no extra vote for generating rows (Figure 3). Unlike SiloFuse, which coordinates a shared latent generator across silos [15], this pattern needs no cross-site generator training; each site can

TABLE III
EVALUATION DATASETS, ALL SMALL IN THE SENSE OF DEFINITION 1. d: FEATURES; $n_{\min}$: ROWS IN THE LEAST FREQUENT CLASS (SHARE IN PARENTHESES); EPV $= n_{\min}/d$; MISS.: SHARE OF EMPTY CELLS.

Dataset	n	d	Classes	$n_{\min}$ (%)	EPV	Miss. (%)
Heart disease	303	13	5	13 (4.3)	1.0	0.1
Breast cancer	569	30	2	212 (37.3)	7.1	0.0
Heart failure	299	12	2	96 (32.1)	8.0	0.0
Diabetes	768	8	2	268 (34.9)	33.5	0.0
Liver disease	583	10	2	167 (28.6)	16.7	0.1
Parkinson's	195	22	2	48 (24.6)	2.2	0.0

also reject augmentation when paired local validation does not improve utility.

This pattern avoids centralizing data but is not by itself a privacy guarantee: a remote LLM receives prompt content, so a sensitive deployment needs an on-premises model or a data-processing agreement, and synthetic rows and updates can both leak information. Our experiments evaluate the local synthesis component one site at a time; end-to-end federated evaluation is future work.

V. EXPERIMENTAL METHODOLOGY

Section IV guarantees validity and volume by construction; whether the resulting tables are faithful, private enough and useful is an empirical question. We organize the evaluation around five research questions, each tied to one or more objectives of Equation (1).

RQ1 How hard and how variable are the real-only baselines?

RQ2 What fidelity, privacy diagnostics and utility does the reference configuration achieve?

RQ3 Which control components change the outcome beyond run-to-run noise?

RQ4 How much does the LLM backend matter?

RQ5 How does LLM synthesis compare with non-LLM generators in cost and outcome?

A. Datasets and Repeated Holdouts

Table III summarizes the six medical classification tables. Five come from the UCI repository [39]–[43]; the diabetes table is the Pima Indians benchmark [44]. We keep the original five-class heart-disease target. Missing values are mean-imputed for numerical columns and mode-imputed for categorical columns.

Each dataset-configuration pair uses ten seeded 80/20 holdouts (seeds 43 to 52). These are repeated holdouts, not disjoint folds. In each repetition, 20% of the training partition is sampled as candidate LLM context and the requested synthetic volume equals the real training size. Generation uses temperature 0.3 and chunks of 30 rows. The same real test partition is shared by three training regimes within a repetition: real only, generated only, and real plus generated. The ten downstream classifiers are random forest (RF), decision tree (DT), multilayer perceptron (MLP), support vector machine (SVM), logistic regression (LR), k-nearest neighbors (kNN), Gaussian naive Bayes (NB), gradient boosting (GB), AdaBoost and XGBoost. We use binary F1 for binary tasks and prevalence-weighted F1 for the multiclass heart-disease task. Binary F1 scores the class coded 1, which is the minority class for heart failure and diabetes but the majority class for liver disease and Parkinson's. The XGBoost evaluation on liver disease failed in every repetition because XGBoost requires class labels $0, \ldots, k-1$ and the liver target is coded $\{1, 2\}$. The released code now re-encodes labels for XGBoost, but filling the cell retroactively would require regenerating the synthetic tables of every configuration, so we report 59 of 60 classifier-dataset cells rather than mix runs from different code versions.

B. Configurations

The analysis admits only configurations that completed all six datasets and all ten repetitions. Eight *architecture* arms share the GPT-4o mini backend: the reference system; no inferred type rules (the integrity validator is disabled); no post-processing; no context join; dense context; diverse context; LLM-selected hyperparameters, in which the LLM chooses context size, selection strategy and temperature from a dataset profile; and SMOTE [6], which uses no LLM. Four *backend* arms run the reference configuration with GPT-4o mini, GPT-4.1 mini, GPT-4o and GPT-4.1. The four models form a 2×2 design that crosses model generation (GPT-4o versus GPT-4.1) with model size (full versus mini). Drawing all of them from one provider is deliberate: it holds the serving interface constant (the same JSON output mode, context-window reporting and token accounting), so differences between arms can be attributed to the model rather than to the API, tokenizer or parsing behavior of different vendors. GPT-4o mini, the lowest-cost model of the four, serves as the reference so that every architecture ablation is run at the cheapest operating point. The controller itself is provider-agnostic: it talks to any OpenAI-compatible endpoint, including locally served open-weight models. Three *non-LLM generators* use the same holdouts: SMOTE, which is also ranked with the architecture arms, and CTGAN and TVAE [1] from the reference `ctgan` implementation with default settings (300 epochs), fitted per holdout on the training split and sampled to the same volume as the LLM arms. All three fit on the mean- and mode-imputed training split, since none accepts missing values; SMOTE balances every class to the majority count and therefore emits only non-majority rows. In total, 13 configurations yield 780 dataset-configuration repetitions and up to 23,400 downstream fits.

Three architecture arms are functionally inert in this setting. With no user rules supplied, disabling post-processing removes a no-op. The context-join flag only changes a legacy blended-fidelity corpus, not the generated-only metrics or the real-plus-generated training set. Disabling the integrity validator cannot change the admitted rows when the reference validator never fires, and it recorded no violations in any reported run. We keep these arms because they are valuable: together with the reference system, they are four independent runs of

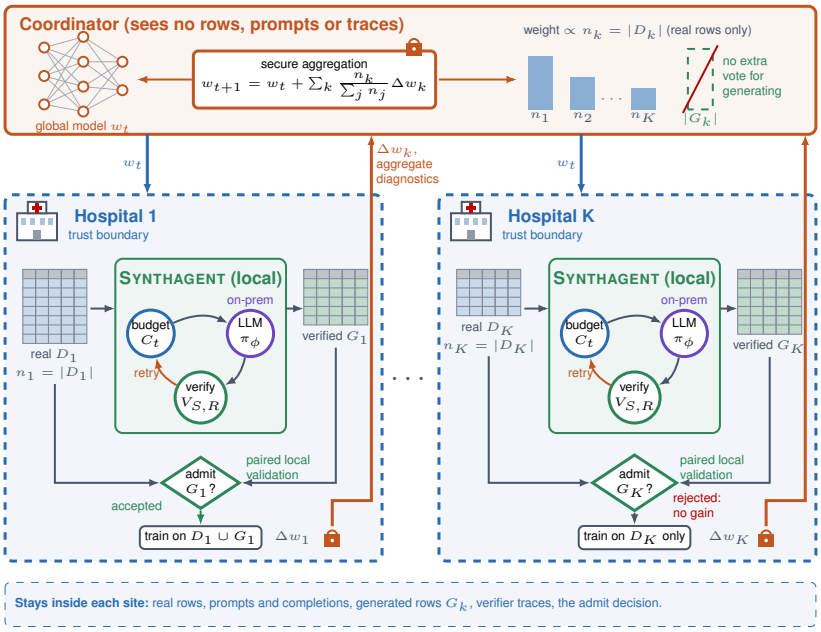


Fig. 3. Cross-silo deployment. Each hospital runs the complete SYNTHAGENT loop (budget, on-premises LLM, verifier with retry) inside its own trust boundary, then decides with paired local validation whether to admit the verified rows G_k (Hospital 1) or to train on real data only (Hospital K). Only the protected update Δw_k and approved aggregate diagnostics cross the boundary. The coordinator weights each update by the real sample count $n_k = |D_k|$ (Equation (7)), so generating more rows buys a site no extra vote.

functionally the same pipeline, and their spread is an empirical estimate of run-to-run variation, which we call the *noise floor*.

C. Metrics

We report generated-only metrics, which compare the synthetic rows alone with the real training rows, so that fidelity is not inflated by real context rows. For numerical columns $j \in \mathcal{J}_{\text{num}}$, marginal fidelity and correlation stability are

$$F_{\text{JSD}} = 100\Big(1 - \frac{1}{|\mathcal{J}_{\text{num}}|} \sum_{j \in \mathcal{J}_{\text{num}}} D_{\text{JS}}(D_j, G_j)\Big), \qquad F_{\text{corr}} = 100\big(1 - \text{mean}\,|\Sigma_D - \Sigma_G|\big), \tag{8}$$

where Σ_D and Σ_G are the correlation matrices of real and generated rows. Both scores equal 100 for identical marginals or correlation matrices. Completeness is the percentage of non-null generated cells. The classifier two-sample test (C2ST) trains a depth-limited random forest to separate real training rows from generated rows; accuracy near 0.5 is desirable [27]. The membership diagnostic scores each real row by the negative distance to its nearest generated row and reports member-versus-holdout ROC AUC, where 0.5 is chance. Cost is measured by total tokens and LLM generation time per repetition.

For metric $m \in \{\text{accuracy}, \text{F1}\}$, dataset d, classifier c and repetition h, the paired augmentation effect is

$$\Delta m_{d,c,h} = m\big(D^{\text{tr}}_{d,h} \cup G_{d,h};\, c\big) - m\big(D^{\text{tr}}_{d,h};\, c\big), \tag{9}$$

where both models are scored on the same holdout $D^{\text{te}}_{d,h}$. Equation (9) holds the test set fixed within a pair, so test-set variation is not mistaken for an augmentation effect. Utility changes are reported in percentage points (pp).

D. Statistical Analysis

Repetitions and classifiers are not independent datasets. To compare configurations, we therefore average over repetitions and classifiers within each dataset and treat the six datasets as matched blocks. Let $\text{rk}_{d,a}$ be the rank of configuration a on dataset d by mean augmented-training score, $\bar{\text{rk}}_a$ its average rank, M the number of datasets and Q the number of configurations. Because all configurations share the same seeded holdouts, ranking augmented scores is equivalent to ranking paired changes up to classifier-level randomness. The Friedman statistic is

$$\chi^2_F = \frac{12M}{Q(Q+1)} \sum_{a=1}^{Q} \bar{\text{rk}}_a^{\,2} - 3M(Q+1). \tag{10}$$

We follow it with two-sided Wilcoxon signed-rank tests of the best average-rank configuration against every other, with Holm correction [45], [46]. Per-cell augmented-versus-real Wilcoxon tests over the ten repetitions are Holm-corrected within each metric. With six blocks, power is limited, so we report ranks and effect sizes next to p-values and interpret effects relative to the noise floor.

VI. RESULTS

We answer the research questions in order. RQ1 establishes the yardstick that every later effect is compared against, RQ2 characterizes the reference system, and RQ3 to RQ5 change one factor at a time: the control components, the backend and the generator family.

A. RQ1: Real-Only Baselines and Small-Sample Variability

Table IV reports every real-only baseline as mean and standard deviation over the ten holdouts. These numbers are the yardstick for every synthetic-data effect that follows. Breast cancer is easy and stable, leaving little headroom. The five-class, severely imbalanced heart-disease task has the lowest baselines. The 195-row Parkinson's table exposes strong learner sensitivity: MLP accuracy is 72.3 ± 18.2 while

tree ensembles exceed 92%. On heart failure, SVM and kNN reach moderate accuracy (63.3% and 61.3%) but near-zero F1 (0.0 and 0.7), because they almost never predict the minority class. Accuracy alone would hide exactly the failure that augmentation is meant to address.

Averaged over classifiers and datasets, real-only accuracy is 75.4%, but this scalar hides that dataset, learner, class balance and split jointly determine the baseline. All augmentation effects below are therefore paired with the matching classifier, dataset and holdout.

Takeaway. Real-only performance already varies by up to 18 points across holdouts, so a synthetic-data effect is meaningful only when it is paired and compared with this spread.

B. RQ2: Reference Configuration

Table V reports the reference configuration (GPT-4o mini). The generated tables are 99.7% complete, and generated-only marginal fidelity averages 72.7 out of 100, ranging from 65.2 on the five-class heart-disease table to 80.2 on diabetes. The membership attack is near chance on average (AUC 0.526) and stays at or below 0.567 on every dataset; it is a diagnostic for this attack, not a privacy guarantee. The C2ST accuracy of 0.878 shows that a classifier can still separate real from generated rows, which is expected for in-context generation from a few dozen demonstrations and is one reason we do not claim that the generated rows are interchangeable with real ones.

Downstream utility is heterogeneous. Averaged over classifiers, augmentation leaves liver disease and diabetes accuracy essentially unchanged ($+0.19$ and -0.03 points), raises diabetes F1 by $+2.39$ points and lowers accuracy on the two heart tables. Figure 4 resolves this by classifier. Accuracy improves in 18 of 59 classifier-dataset cells, decreases in 38 and is unchanged in three; F1 improves in 26, decreases in 31 and is unchanged in two. The largest gains are concentrated where real-only training fails on the minority class: kNN on heart failure, whose real-only F1 is 0.7%, gains $+15.3$ F1 points, and MLP on Parkinson's, the most unstable baseline, gains $+8.2$ accuracy points. After Holm correction over 59 tests per metric, no individual cell is significant at 0.05.

Figure 5 adds holdout-level uncertainty. Each point averages the paired change over classifiers within one seeded holdout; diamonds show the ten-holdout mean with 95% t intervals. Accuracy is consistently negative for heart disease and heart failure, while liver disease stays centered on zero. Diabetes is the only dataset with a clearly positive mean F1 change at neutral accuracy. Parkinson's combines a large positive F1 outlier with mostly negative repetitions, illustrating why a single favorable run would be misleading.

Takeaway. The reference system delivers complete tables with near-chance membership-attack performance, but its utility effect is learner-specific: it helps most where real-only training ignores the minority class and slightly hurts already strong learners. The next question is which design choices move these numbers.

C. RQ3: Control Components and the Noise Floor

Table VI compares the eight architecture arms. The first four rows (shaded block) are functionally the same pipeline, so their spread is the noise floor: JSD 72.43–72.92, correlation 70.83–72.93, mean accuracy change -1.80 to -1.19 points, mean F1 change -0.18 to $+0.28$ points and MIA AUC 0.523–0.534. Two consequences follow. First, differences of about 0.5 JSD points, 2 correlation points or 0.6 utility points between two runs of an LLM pipeline are not evidence of a design effect. Second, latency is not a reliable cost signal: the four equivalent runs take between 228 and 379 seconds per repetition, a $1.7\times$ spread driven by the provider rather than the design, while token usage varies by less than 2% (72,188–73,510). We therefore treat tokens as the primary cost measure.

Against this floor, the context strategies have small effects. Dense context stays inside the floor on every metric. Diverse context is the only control choice that leaves the floor on fidelity: it gives the highest JSD (73.27) and correlation (73.72) and the lowest C2ST accuracy (0.872), but also the highest MIA AUC (0.544). This is the trade-off one would expect from selecting demonstrations that cover the feature space: generated rows land closer to more real records, which helps fidelity and slightly helps the membership attack. LLM-selected hyperparameters stay inside the floor on fidelity and utility, fall below it on correlation (70.10) and cost 57% more tokens (115,245 versus 73,510), so delegating configuration to the LLM does not pay off in this study.

The Friedman tests confirm that the architecture arms are indistinguishable in utility: $\chi^2_F(7) = 3.10$, $p = .875$ for accuracy and $\chi^2_F(7) = 4.39$, $p = .734$ for F1, and every Holm-adjusted post-hoc comparison has $p_{\mathrm{Holm}} = 1.0$. Figure 6 shows the average ranks. LLM-selected hyperparameters rank best on accuracy (3.75) and the inert no-post-processing arm best on F1 (3.33), which itself illustrates how little the ranks separate: an arm that changes nothing ranks first. SMOTE ranks sixth of eight on accuracy (5.00) but second on F1 (3.67). Figure 7 summarizes the multi-objective picture: diverse context leads on fidelity, the reference leads on mean accuracy change among the LLM arms, and SMOTE leads on cost, correlation and mean F1 change but ranks last on JSD and accuracy.

What, then, do the control components contribute? Their value lies in reliability and observability rather than in shifting the fidelity mean, and the experiment reflects this. Context budgeting and rotation let the same 20% candidate pool fit every backend's window without manual tuning. The parser, repair path and validator are safety mechanisms, and the backends exercise them differently. The reference GPT-4o mini runs recorded no JSON repair and no rule violation. GPT-4.1 mini, the fastest backend, recorded on average 7.4 typed-rule violations per repetition on breast cancer, all routed to targeted correction, and needed one JSON repair across its ten diabetes repetitions. Its tables were still 100% complete with the best mean accuracy change of any configuration, which is the behavior the verifier is designed to provide: a faster

TABLE IV

REAL-ONLY BASELINES (RQ1) OVER TEN REPEATED 80/20 HOLDOUTS, AS MEAN IN PERCENT WITH THE STANDARD DEVIATION IN SMALL TYPE. XGBOOST ON LIVER DISEASE (–) IS MISSING BECAUSE THAT TARGET IS CODED $\{1, 2\}$ WHILE THE XGBOOST VERSION USED REQUIRES LABELS $\{0, 1\}$, SO ALL TEN FITS FAILED; WE REPORT THE GAP RATHER THAN IMPUTE IT.

	Heart dis.		Breast		Heart fail.		Diabetes		Liver		Parkinson's	
Classifier	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
RF	61.0 $\pm$4.4	57.0 $\pm$5.9	95.6 $\pm$1.8	95.0 $\pm$2.8	84.3 $\pm$3.3	76.8 $\pm$4.4	74.9 $\pm$2.8	60.4 $\pm$6.0	71.1 $\pm$3.5	81.4 $\pm$2.4	92.3 $\pm$3.2	94.8 $\pm$2.2
DT	53.4 $\pm$3.9	54.0 $\pm$5.0	93.3 $\pm$2.3	92.1 $\pm$4.0	79.2 $\pm$4.7	68.4 $\pm$9.2	71.1 $\pm$3.8	57.2 $\pm$6.4	66.0 $\pm$3.2	76.3 $\pm$3.1	85.1 $\pm$5.6	89.7 $\pm$3.8
MLP	56.6 $\pm$6.4	52.5 $\pm$7.6	94.7 $\pm$0.9	94.0 $\pm$2.1	48.3 $\pm$12.9	34.2 $\pm$24.7	68.4 $\pm$4.1	52.9 $\pm$12.0	68.4 $\pm$4.1	79.4 $\pm$3.6	72.3 $\pm$18.2	77.8 $\pm$26.1
SVM	54.8 $\pm$6.8	39.0 $\pm$7.9	93.3 $\pm$2.1	92.4 $\pm$3.4	63.3 $\pm$5.2	0.0 $\pm$0.0	75.6 $\pm$2.2	57.3 $\pm$4.7	70.7 $\pm$2.9	82.8 $\pm$2.0	81.0 $\pm$8.2	88.5 $\pm$5.2
LR	59.8 $\pm$5.6	56.5 $\pm$5.0	95.2 $\pm$2.0	94.6 $\pm$2.6	80.5 $\pm$3.6	68.9 $\pm$8.4	77.3 $\pm$2.0	62.8 $\pm$5.4	70.3 $\pm$4.0	81.1 $\pm$3.2	86.9 $\pm$5.6	91.3 $\pm$3.9
kNN	55.4 $\pm$6.3	44.4 $\pm$7.9	94.3 $\pm$1.8	93.4 $\pm$3.2	61.3 $\pm$4.1	0.7 $\pm$2.1	75.5 $\pm$1.8	60.0 $\pm$3.3	68.6 $\pm$4.2	79.7 $\pm$3.2	83.6 $\pm$5.6	89.5 $\pm$3.9
NB	55.7 $\pm$5.3	56.9 $\pm$5.0	93.9 $\pm$1.7	93.1 $\pm$3.2	75.0 $\pm$4.9	56.7 $\pm$13.1	76.4 $\pm$2.4	64.0 $\pm$4.9	55.9 $\pm$3.7	55.9 $\pm$3.8	71.5 $\pm$4.4	76.6 $\pm$4.0
GB	54.1 $\pm$5.5	52.8 $\pm$6.3	95.9 $\pm$2.2	95.4 $\pm$2.8	83.0 $\pm$4.9	75.4 $\pm$7.2	74.7 $\pm$3.0	61.7 $\pm$5.8	67.9 $\pm$3.3	78.7 $\pm$2.4	92.1 $\pm$4.0	94.7 $\pm$2.6
AdaBoost	60.8 $\pm$7.6	60.7 $\pm$7.3	97.1 $\pm$1.5	96.8 $\pm$1.7	82.5 $\pm$3.0	74.2 $\pm$6.5	74.9 $\pm$2.2	60.8 $\pm$4.7	67.4 $\pm$3.4	78.2 $\pm$3.3	91.3 $\pm$3.8	94.2 $\pm$2.5
XGBoost	54.3 $\pm$4.6	52.0 $\pm$5.1	97.2 $\pm$1.4	96.9 $\pm$1.8	83.5 $\pm$4.0	75.5 $\pm$7.0	72.7 $\pm$2.8	59.1 $\pm$6.2	–	–	93.3 $\pm$5.2	95.6 $\pm$3.2

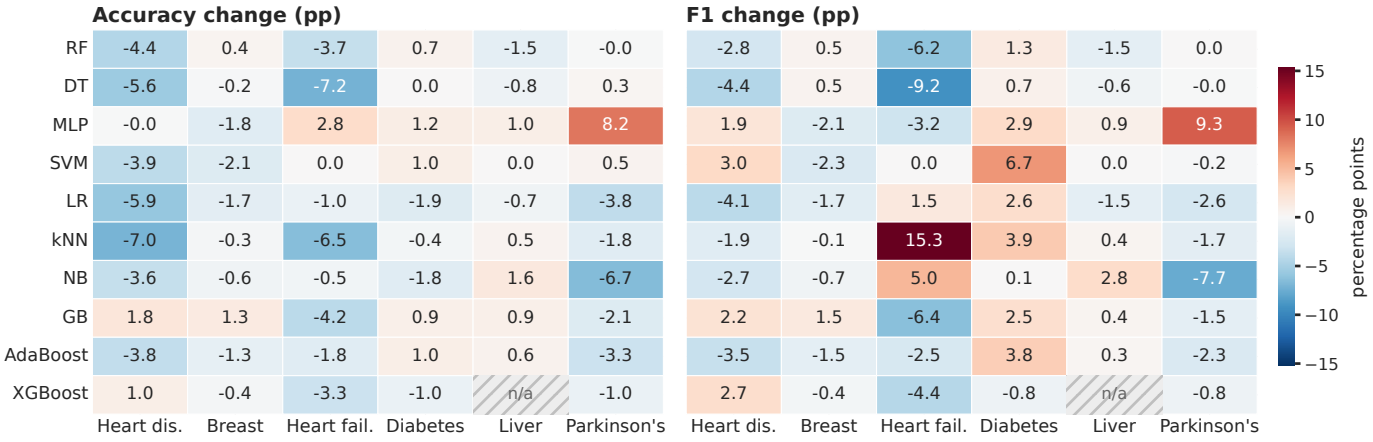


Fig. 4. Change from real-only to real-plus-generated training for the reference configuration (RQ2), per classifier and dataset. Hatched cells (n/a) were not evaluated. The same generator can help one learner and hurt another, which is why we report the full matrix.

TABLE V

REFERENCE CONFIGURATION (RQ2): GPT-4O MINI, AVERAGED OVER TEN REPETITIONS AND ALL AVAILABLE CLASSIFIERS. JSD AND CORRELATION ARE GENERATED-ONLY SCORES ON A 0–100 SCALE. ACCURACIES ARE IN PERCENT; UTILITY CHANGES ARE IN PERCENTAGE POINTS RELATIVE TO REAL-ONLY TRAINING.

	Generated only				Acc. (%)		Change (pp)	
Dataset	JSD	Corr.	C2ST	MIA	Real	Aug.	Acc.	F1
Breast	77.2	74.5	0.911	0.550	95.1	94.4	−0.66	−0.62
Diabetes	80.2	44.0	0.897	0.504	74.2	74.1	−0.03	+2.39
Heart dis.	65.2	76.6	0.893	0.507	56.6	53.4	−3.15	−0.97
Heart fail.	71.2	81.2	0.892	0.534	74.1	71.6	−2.53	−1.00
Liver	71.8	73.8	0.883	0.495	67.4	67.6	+0.19	+0.13
Parkinson's	70.6	85.3	0.789	0.567	84.9	84.0	−0.97	−0.77
Mean	72.7	72.6	0.878	0.526	75.4	74.2	−1.19	−0.14

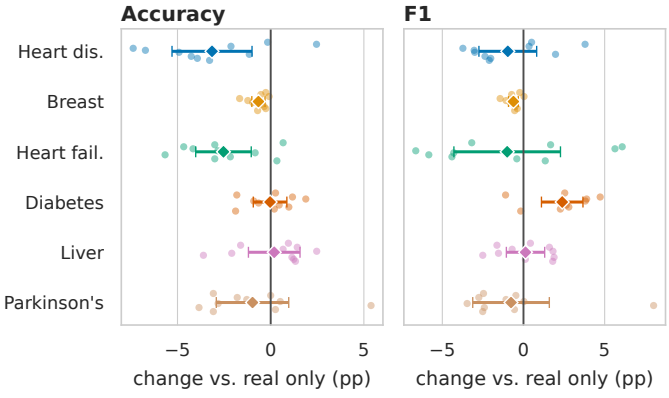


Fig. 5. Repeated-holdout uncertainty for the reference configuration (RQ2). Points are seeded holdouts after averaging paired changes over classifiers; diamonds are means and bars are 95% t intervals. The plot is descriptive: the six datasets, not the holdouts, are the independent blocks in the configuration tests.

but less disciplined backend can be used safely because its defects are caught and repaired rather than silently admitted. Exact-volume completion turns under-generation from a silent defect into recorded state. A dedicated stress test with weaker or local models, stricter rules and boundary enforcement is needed to measure how often each safeguard fires.

Takeaway. Control components buy reliability, not fidelity. Within one backend, only diverse context moves outside the noise floor, and only slightly. We therefore turn to the one

TABLE VI

ARCHITECTURE ARMS (RQ3), ALL WITH THE GPT-4O MINI BACKEND. THE FIRST FOUR ROWS ARE FUNCTIONALLY EQUIVALENT AND DEFINE THE NOISE FLOOR. SECONDS AND TOKENS ARE MEANS PER REPETITION. BOLD MARKS THE BEST VALUE AMONG THE LLM ARMS; LOWER IS BETTER FOR C2ST, FOR THE DISTANCE OF MIA AUC FROM 0.5, AND FOR COST.

	Generated-only fidelity and privacy				Utility change (pp)		Cost per repetition	
Configuration	JSD	Corr.	C2ST	MIA AUC	Acc.	F1	Seconds	Tokens
Noise floor: functionally equivalent runs of the reference pipeline								
Reference SYNTHAGENT	72.70	72.57	0.878	0.526	−1.19	−0.14	378.7	73,510
No inferred type rules	72.92	72.93	0.876	0.534	−1.80	−0.04	321.6	**72,188**
No post-processing	72.43	70.83	0.883	**0.523**	−1.71	+0.28	251.5	72,235
No context join	72.52	71.18	0.880	0.531	−1.31	−0.18	**228.1**	72,576
Active design choices								
Dense context	72.87	70.88	0.878	0.525	−1.24	+0.07	238.8	72,600
Diverse context	**73.27**	**73.72**	**0.872**	0.544	−1.34	−0.13	271.1	73,757
LLM-selected hyperparameters	72.53	70.10	0.886	0.526	−1.25	+0.11	310.2	115,245
SMOTE†	69.93	85.02	0.795	0.532	−3.08	+0.33	0.015	0

†SMOTE emits only non-majority rows, so its generated-only metrics describe that tail and are not directly commensurate with full-table generators. Table VIII compares it with CTGAN and TVAE.

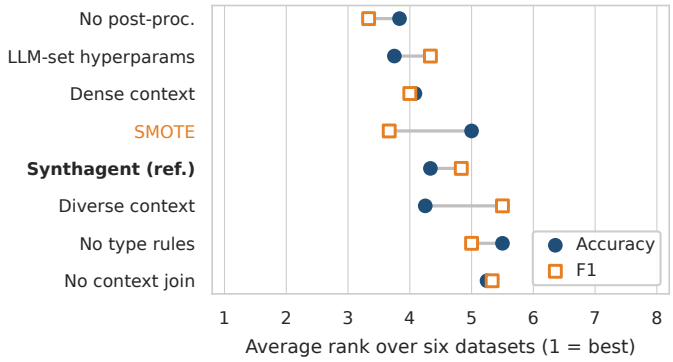


Fig. 6. Average ranks of the architecture arms across six matched datasets (lower is better). Neither accuracy nor F1 shows a significant configuration effect.

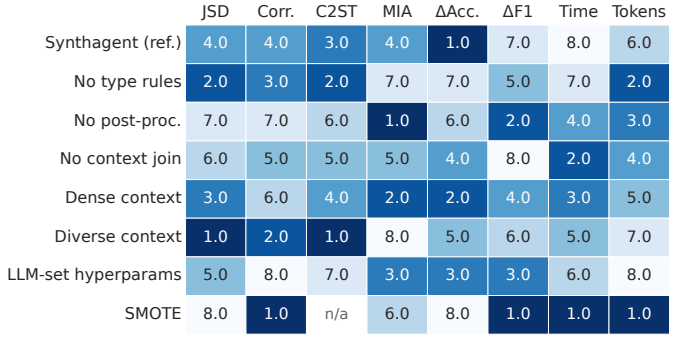


Fig. 7. Multi-objective rank scorecard for the architecture arms (rank 1 is best). C2ST and MIA are ranked by distance from 0.5. The SMOTE C2ST rank is omitted because its minority-only output makes raw C2ST accuracy non-commensurate. Ranks describe trade-offs and do not replace the significance tests.

factor not yet varied: the model behind the controller.

D. RQ4: Backend Choice

Backend choice moves the outcome more than any control toggle (Table VII, Figure 8). All three alternative backends

TABLE VII

LLM BACKENDS (RQ4) UNDER THE REFERENCE CONFIGURATION. SECONDS AND TOKENS ARE MEANS PER REPETITION.

Backend	JSD	Corr.	ΔAcc.	ΔF1	Sec.	Tokens
GPT-4o mini	72.70	**72.57**	−1.19	−0.14	378.7	**73,510**
GPT-4.1 mini	73.60	62.82	−0.23	+0.92	**168.5**	72,069
GPT-4o	73.95	64.32	−0.50	+0.25	255.2	80,046
GPT-4.1	**75.72**	63.30	−0.44	+1.09	201.7	80,652

exceed the noise floor in accuracy change, and two of them in F1 change. GPT-4.1 gives the highest marginal fidelity (75.72, 2.8 points above the floor) and the largest mean F1 gain (+1.09 points). GPT-4.1 mini gives the best mean accuracy change (−0.23 points, nearly one point better than the reference) and an F1 gain of +0.92 points, with the shortest generation time. The gains come with a consistent cost in correlation preservation: all three alternatives score 62.8–64.3, more than six points below the lowest equivalent run of GPT-4o mini (70.83). Stronger backends therefore reproduce marginal distributions and label-relevant structure better but pairwise dependencies less faithfully, a trade-off that a practitioner can now see before choosing a backend. None of these differences is statistically established across six datasets: $\chi^2_F(3) = 2.00$, $p = .572$ for accuracy and $\chi^2_F(3) = 1.80$, $p = .615$ for F1. The closest post-hoc comparison, GPT-4.1 mini versus GPT-4o mini in accuracy, differs by +0.86 points in mean augmented accuracy with $p_{\text{Holm}} = .188$.

Takeaway. The backend is the main lever on fidelity and utility, and it trades marginal fidelity against correlation preservation. Because the controller is backend-agnostic, switching models requires no change to the pipeline.

E. RQ5: LLM Synthesis Versus Non-LLM Generators

Table VIII places the reference and the strongest backend next to three generators that need no LLM. Cost separates them first. SMOTE needs 0.015 seconds per repetition; TVAE and CTGAN fit in 5.6 and 14.6 seconds of CPU time; the LLM

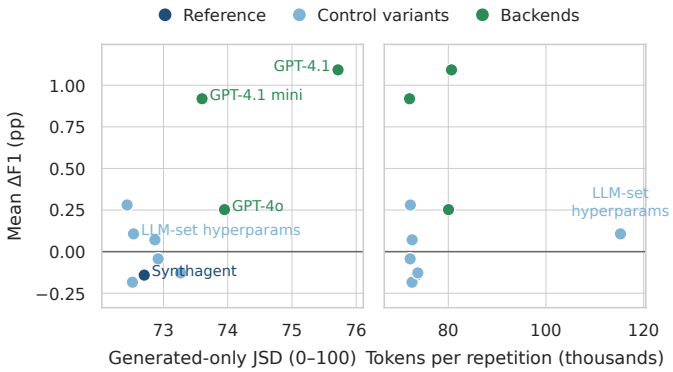


Fig. 8. Fidelity, utility and token cost of the complete LLM arms (RQ3, RQ4). Control variants cluster inside the noise floor; backends spread out along fidelity and F1. The non-LLM generators use no tokens and are compared in Table VIII.

TABLE VIII
GENERATORS COMPARED ON THE SAME HOLDOUTS (RQ5). GENERATED-ONLY JSD AND CORRELATION ARE ON A 0–100 SCALE; UTILITY CHANGES ARE IN PERCENTAGE POINTS. SECONDS ARE API WALL-CLOCK FOR THE LLM ARMS AND CPU FIT-AND-SAMPLE TIME FOR THE OTHERS.

Generator	JSD	Corr.	C2ST	MIA	Valid	ΔAcc.	ΔF1	Sec.	Tok.
SYNTHAGENT, 4o mini	72.7	72.6	0.878	0.526	–‡	−1.19	−0.14	379	73.5k
SYNTHAGENT, 4.1	75.7	63.3	0.877	0.527	–‡	−0.44	+1.09	202	80.7k
SMOTE†	69.9	85.0	0.795	0.532	**85.7**	−3.08	+0.33	**0.01**	0
CTGAN	**77.6**	75.2	0.936	**0.503**	29.6	−1.40	−3.37	14.6	0
TVAE	71.9	**85.6**	0.868	0.515	75.6	−0.09	−0.21	5.6	0

†Non-majority rows only; its C2ST is not commensurate. Valid: % of generated rows inside the observed schema (all columns present, numbers integral where the column is and within the observed range, categories observed). ‡Not logged for the LLM runs.

arms spend 73,000 to 81,000 tokens and several minutes of API time, but no per-table training.

No generator wins on every axis. CTGAN reaches the highest marginal fidelity (JSD 77.6) but produces the most detectable rows: a classifier separates them from real rows with 0.936 accuracy, against 0.877 for the LLM arms. TVAE and SMOTE preserve correlations best (85.6 and 85.0), as expected from models that stay close to the training rows, and GPT-4.1 trades correlation (63.3) for marginal fidelity (75.7). All five stay near chance for the membership attack (0.503–0.532). Schema validity separates the fitted generators sharply. Only 29.6% of CTGAN rows fall inside the observed schema (1.0% on Parkinson's, whose 22 continuous columns each give a chance to leave the observed range); TVAE keeps 75.6% (58.2–88.0% per table). SMOTE is fully valid on five tables but only 14.4% valid on heart disease, where interpolation produces fractional values in integer-coded columns such as `ca` and `thal`.

Utility follows the class structure of each table. SMOTE gives the largest gains where the scored class is a rare outcome, $+10.91$ F1 points on heart failure and $+4.11$ on diabetes, but rebalancing costs accuracy everywhere (-3.08 points on average, -7.43 on the five-class heart-disease table) and loses 8.20 F1 points on liver disease, where the scored class is the majority. CTGAN loses F1 on average (-3.37), driven by heart failure (-10.65). TVAE is the most conservative generator: it gives the best mean accuracy change (-0.09) and moves no dataset's mean F1 by more than 3 points. Under SYNTHAGENT, GPT-4.1 gives the largest mean F1 gain of all five ($+1.09$, with $+6.35$ on heart failure) while keeping every dataset within 6.4 F1 points. Across datasets, the differences are not significant: $\chi^2_F(4) = 6.80$, $p = .147$ for accuracy and $\chi^2_F(4) = 5.60$, $p = .231$ for F1, with every Holm-adjusted comparison at $p \geq .25$; TVAE has the best accuracy rank (2.00) and the two LLM arms share the best F1 rank (2.50).

Takeaway. The LLM path is not a free improvement over classical generators: TVAE matches it on accuracy in seconds, and SMOTE remains the cheapest fix for a known rare positive class. What the LLM path adds is training-free synthesis with the best mean F1 change, rows that are harder to tell from real ones than CTGAN's, no large per-dataset failures, and, through the control plane, schema validity, exact volume and an audit trail that none of the fitted generators provides.

VII. DISCUSSION AND LIMITATIONS

What the evidence supports: SYNTHAGENT is best understood as a control plane for LLM synthesis. It does not make an LLM a universally better augmenter, and our data say so plainly. What it does is make syntax, schema, volume, cost, fidelity, privacy diagnostics and utility explicit, inspectable state. This has three practical consequences. Generated tables arrive complete and at the requested size. Their properties are measured on the same paired protocol as the real-only baseline, so a team can see whether augmentation helps a given learner before using it. And design decisions can be checked against a measured noise floor rather than against a single favorable run.

Lessons for practitioners: (1) Choose the backend first: it is the only factor in this study whose effect clearly exceeds run-to-run noise. (2) Track tokens, not wall-clock time, as the cost of an LLM pipeline. (3) Expect augmentation to help most where a real-only learner collapses on the minority class, and verify it per learner. (4) Do not delegate configuration to the LLM without measuring the extra cost.

Threats to validity: Only six small medical datasets act as independent blocks, which limits the power of the rank tests. The repeated holdouts are not stratified. The Parkinson's table contains repeated recordings per subject without a subject identifier, so one person's recordings can appear in both train and test. Schema and value ranges are inferred once from the full table; this exposes holdout extrema to the profiler and should be replaced by train-only profiling in a strict predictive evaluation. JSD uses fixed histograms and is dominated by numerical columns. The membership attack is a single distance-based adversary over numerical features, and no result establishes differential privacy. The reported runs never triggered the repair and correction paths, so their benefit under weaker models remains to be measured; in the current implementation, the output of the last correction round is admitted without a further validation pass, and rows obtained in top-up rounds

are parsed but not re-validated. Context selection is not class-stratified, so rare classes can be absent from a prompt; a label-aware policy with minimum per-class coverage is a natural next step. All backends come from a single provider, so the results may not transfer to other model families; in particular, locally served open-weight models, which matter most for on-premises clinical deployment, remain to be evaluated through the same OpenAI-compatible interface. Two further OpenAI models, GPT-4.1 nano and o4-mini, were started but did not complete every dataset and are excluded by the completeness gate. CTGAN and TVAE run with library defaults; tuned deep generators, or diffusion models such as TabDDPM, may do better on some tables. Finally, provider-side model revisions are not pinned beyond the model identifiers stored with each run.

Societal impact: Synthetic medical rows can ease development, teaching and pipeline testing, but plausible records can be mistaken for clinical evidence and must not inform diagnosis or policy without domain review. Near-chance performance against one membership attack does not justify releasing sensitive data; privacy-critical deployments need threat modeling, stronger attacks and a formal privacy mechanism.

Reproducibility: The implementation, the ablation runner and the asset generator are publicly available.[1] The script `generate_paper_assets.py` reads the checkpoint, admits only configurations that completed every dataset and repetition, recomputes utility from the per-repetition files, runs the statistical tests and regenerates every table and figure in this paper.

VIII. Conclusion

We presented SYNTHAGENT, a verifier-guided control plane that turns untrusted LLM completions into complete, schema-valid and auditable small tables. On six medical tables and 780 repetitions, it produced complete tables with stable fidelity and near-chance membership-attack performance. A noise floor built from functionally equivalent runs shows that context and control toggles stay within run-to-run variation, while the LLM backend is the factor that moves fidelity and utility. No generator, the non-LLM baselines included, is a significant utility winner across datasets, so synthetic volume is not a substitute for evidence; the value of the control plane is to make generation choices, failures, costs and their paired downstream consequences explicit enough to decide, per task and per learner, when augmentation is warranted.

Acknowledgment

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 – Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025.

[1]TODO(authors): add the public repository URL required by the ICDE artifact policy.

References

[1] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[2] P. Peduzzi, J. Concato, E. Kemper, T. R. Holford, and A. R. Feinstein, "A simulation study of the number of events per variable in logistic regression analysis," *Journal of Clinical Epidemiology*, vol. 49, no. 12, pp. 1373–1379, 1996.

[3] R. D. Riley, J. Ensor, K. I. E. Snell, F. E. Harrell, G. P. Martin, J. B. Reitsma, K. G. M. Moons, G. Collins, and M. van Smeden, "Calculating the sample size required for developing a clinical prediction model," *BMJ*, vol. 368, p. m441, 2020.

[4] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[5] N. Seedat, N. Huynh, B. van Breugel, and M. van der Schaar, "Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.

[6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[7] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 17 564–17 579.

[8] H. Zhang, J. Zhang, Z. Shen, B. Srinivasan, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=4Ay23yeuz0

[9] A. V. Solatorio and O. Dupriez, "REaLTabFormer: Generating realistic relational and tabular data using transformers," *arXiv preprint arXiv:2302.02041*, 2023.

[10] J. Kim, T. Kim, and J. Choo, "EPIC: Effective prompting for imbalanced-class data synthesis in tabular data classification via large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[11] A. Sanghi and J. R. Haritsa, "Synthetic data generation for enterprise DBMS," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3585–3588.

[12] J. Fan, T. Liu, G. Li, J. Chen, Y. Shen, and X. Du, "Relational data synthesis using generative adversarial networks: A design space exploration," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1962–1975, 2020.

[13] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2014.

[14] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1886–1899, 2021.

[15] A. Shankar, H. Brouwer, R. Hai, and L. Y. Chen, "SiloFuse: Cross-silo synthetic data generation with latent tabular diffusion models," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 110–123.

[16] C. Lee, J. Kim, and N. Park, "CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 18 940–18 956.

[17] L. Chen, C. Wu, J. Li, S. Liu, G. Ye, and H. Chai, "TabLoft: Tabular data generation based on LLM with ordered features," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1044–1056.

[18] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[19] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[20] J. Qin, S. Huang, Y. Wang, J. Zhu, Y. Zhang, Y. Miao, R. Mao, M. Onizuka, and C. Xiao, "BClean: A Bayesian data cleaning system," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3407–3420.

[21] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava, and J. Szlichta, "Recovering structure in unstructured LLM outputs," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 4329–4330.

[22] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 13–24.

[23] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3747–3754.

[24] J. Liang, C. Lei, X. Qin, J. Zhang, A. Katsifodimos, C. Faloutsos, and H. Rangwala, "Featpilot: Automatic feature augmentation on tabular data," in *Proceedings of the IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 2148–2160.

[25] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[26] A. D. Lautrup, T. Hyrup, A. Zimek, and P. Schneider-Kamp, "SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data," *Data Mining and Knowledge Discovery*, vol. 39, no. 1, p. 6, 2025.

[27] D. Lopez-Paz and M. Oquab, "Revisiting classifier two-sample tests," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJkXfE5xx

[28] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.

[29] J. Hyeong, J. Kim, N. Park, and S. Jajodia, "An empirical study on the membership inference attack against tabular data synthesis models," *arXiv preprint arXiv:2208.08114*, 2022.

[30] A. Machanavajjhala, J. Gehrke, D. Kifer, and M. Venkitasubramaniam, "ℓ-diversity: Privacy beyond k-anonymity," in *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 2006, p. 24.

[31] N. Li, T. Li, and S. Venkatasubramanian, "t-closeness: Privacy beyond k-anonymity and ℓ-diversity," in *Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE)*, 2007, pp. 106–115.

[32] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.

[33] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Ag"uera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[34] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[35] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[36] N. Rieke, J. Hancox, W. Li, F. Milletar'i, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, and M. J. Cardoso, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.

[37] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.

[38] Q. Chen, Y. Chen, B. Yan, X. Jiang, X. Zhang, Y. Kang, T. Zhang, W. Huang, C. Gao, L. Fan, and Q. Yang, "Model trip: Enhancing privacy and fairness in model fusion across multi-federations for trustworthy global healthcare," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024.

[39] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart disease," UCI Machine Learning Repository, 1988.

[40] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[41] "Heart failure clinical records," UCI Machine Learning Repository, 2020.

[42] B. V. Ramana, M. S. P. Babu, and N. B. Venkateswarlu, "ILPD (indian liver patient dataset)," UCI Machine Learning Repository, 2012.

[43] M. Little, "Parkinsons," UCI Machine Learning Repository, 2007.

[44] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[45] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[46] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.